Hybrid ML-Guided Load-Aware Adaptive CPU Scheduler with xv6 Kernel Implementation

Dev Agarwal (2410110118),
Kavya Pratap Singh Chauhan (2410110167),
Keshav Goyal (2410110170),
Antra Agarwal (2410110404)

Abstract

This paper presents the design and implementation of a Hybrid ML-Guided Load-Aware Adaptive CPU Scheduler (HML-LAS) integrated into the xv6 RISC-V operating system kernel. Traditional CPU scheduling algorithms such as Round Robin (RR), Shortest Job First (SJF), and Multi-Level Feedback Queue (MLFQ) exhibit fundamental limitations including fixed behavior, susceptibility to starvation, and inability to adapt to dynamic workloads. HML-LAS addresses these limitations through a three-layer scheduling framework: (1) a formula-based burst prediction model leveraging process execution history, (2) an Exponential Weighted Moving Average (EWMA) mechanism that dynamically adapts prediction weights based on system load, and (3) a process classification layer that distinguishes I/O-bound from CPU-bound processes to apply differentiated scheduling strategies. Additionally, an aging mechanism prevents starvation by boosting priority for long-waiting processes. The scheduler was implemented by modifying only two files in the xv6 kernel (kernel/proc.h and kernel/proc.c) and was evaluated through a 1000-process simulation. Results demonstrate that HML-LAS achieves an average turnaround time of 3721 ticks, outperforming Round Robin (5192 ticks) and MLFQ (5455 ticks) by approximately 28% and 32% respectively, while maintaining starvation-free operation across all tested workloads.

I. Introduction

CPU scheduling is one of the most critical functions of any operating system. It determines which process is allocated CPU time at any given moment, directly influencing system throughput, responsiveness, fairness, and overall efficiency. The design of an effective scheduling algorithm must balance multiple competing objectives including minimizing average waiting time, preventing starvation, and adapting to dynamically changing workload conditions.

Classical scheduling algorithms have been extensively studied and deployed in real systems. First Come First Serve (FCFS) is simple but suffers from the convoy effect, where long-running processes delay short ones. Shortest Job First (SJF) is theoretically optimal for minimizing average waiting time but is impractical because it requires prior knowledge of burst times. Round Robin (RR) ensures fairness through equal time slices but treats all processes identically regardless of their actual CPU requirements, leading to unnecessary

context switching and poor performance for heterogeneous workloads. Priority-based scheduling introduces differentiation but can cause indefinite starvation of low-priority processes.

More sophisticated approaches such as Multi-Level Feedback Queue (MLFQ) attempt to combine the advantages of multiple strategies but introduce significant complexity and still rely on fixed configuration parameters that may not be optimal across all workload types. The fundamental challenge is that no single static algorithm performs optimally across all real-world scenarios.

This project proposes and implements HML-LAS (Hybrid ML-Guided Load-Aware Adaptive Scheduler), a novel CPU scheduling algorithm that addresses the above limitations. HML-LAS introduces three key innovations: adaptive burst time prediction using process history and EWMA, load-sensitive scheduling decisions, and automatic process classification into I/O-bound and CPU-bound categories. The entire implementation is carried out inside the xv6 RISC-V kernel — a real, functional teaching operating system developed at MIT — demonstrating practical feasibility. The scheduler is evaluated against Round Robin, Dynamic Round Robin, and MLFQ through a 1000-process simulation.

II. Literature Survey

The development of HML-LAS draws upon three key areas of prior work: feedback-control-based scheduling, machine-learning-based scheduling, and optimized Round Robin approaches. The following subsections summarize the most relevant contributions.

A. Feedback-Control Based Scheduling

A highly influential contribution in adaptive scheduling (311 citations) is the feedback-control-based approach introduced by Lu, Stankovic, Tao, and Son [1], which applies PID control theory to CPU scheduling. Traditional schedulers such as Earliest Deadline First (EDF) operate in an open-loop fashion, making no corrections based on observed system behavior. The proposed Feedback Control EDF (FC-EDF) scheduler continuously monitors the deadline miss ratio and dynamically adjusts CPU load using a PID controller, achieving high utilization while maintaining soft performance guarantees.

The key insight from this work is that scheduling can be treated as a closed-loop control system, where real-time feedback enables the scheduler to adapt to workload variations and maintain desired performance levels. This paper provided foundational motivation for the feedback and adaptive mechanisms incorporated in HML-LAS.

B. Machine Learning Based Scheduling

Nemirovsky et al. [2] (83 citations) demonstrated the application of Artificial Neural Networks (ANNs) to predict thread performance on heterogeneous CPU architectures. Their scheduler predicts each thread's behavior during the next scheduling quantum and selects the optimal core mapping to maximize throughput.

Evaluated on SPEC2006 and SPLASH-2 benchmarks using a 1-large 3-small core system, their approach achieved throughput improvements of 25-31% over conventional heterogeneous round-robin schedulers.

This work validates the core premise of HML-LAS: that predictive, learning-based approaches can significantly outperform static scheduling algorithms. The insight that lightweight prediction models are both practical and impactful directly shaped the formula-based burst prediction mechanism in our design.

C. Optimized Round Robin Scheduling

Dash, Sahu, and Samantra [3] (61 citations) proposed the Dynamic Average Burst Round Robin (DABRR) algorithm, which replaces the fixed time quantum of traditional Round Robin with a dynamically calculated quantum based on the average burst time of currently active processes. Experimental evaluation across six test cases showed that DABRR reduces average waiting time by 41% and turnaround time by 31% compared to standard Round Robin, while also reducing context switches.

DABRR establishes an important baseline: even modest adaptations to classical algorithms can yield substantial performance improvements. This motivates the more sophisticated adaptive mechanisms implemented in HML-LAS, which extends beyond simple quantum adjustment to incorporate full burst prediction and process classification.

D. Overall Insights

The literature reveals a clear evolutionary trajectory from static to adaptive to intelligent scheduling. Three consistent conclusions emerge: (1) traditional algorithms are insufficient for dynamic, heterogeneous workloads; (2) feedback and prediction mechanisms substantially improve scheduling outcomes; and (3) even lightweight models can achieve significant gains when designed with practical kernel constraints in mind. These insights directly motivated the design of HML-LAS, which combines prediction, load-awareness, and feedback-driven adaptation into a unified scheduling framework.

III. Proposed Methodology

A. System Setup

The implementation was carried out in the following environment:

Component	Details
Host OS	Ubuntu 22.04 (WSL2 on Windows 11)
Emulator	QEMU (qemu-system-riscv64) v6.2
Target OS	xv6-riscv (MIT Teaching OS)
Compiler	riscv64-linux-gnu-gcc

Architecture	RISC-V 64-bit

B. Algorithm Design — HML-LAS

HML-LAS is a three-layer adaptive scheduling algorithm. Each layer adds a distinct dimension of intelligence over the baseline Round Robin approach.

Layer 1 — Burst Prediction Formula:

The foundation of HML-LAS is a weighted formula that predicts a process's next burst time based on observable runtime metrics:

$$\text{predicted} = (5 \times \text{last\_burst} + 3 \times \text{waiting\_time} + 2 \times \text{load}) / 10$$

The weights reflect the relative importance of each factor: process history (weight 0.5) is the strongest predictor of future behavior; waiting time (weight 0.3) ensures long-waiting processes receive gradually increasing priority; and system load (weight 0.2) adjusts predictions to reflect congestion levels.

Layer 2 — EWMA Adaptive Weighting:

The second layer replaces the fixed weight for last_burst with a load-adaptive parameter α (alpha), implementing an Exponential Weighted Moving Average (EWMA) scheme:

$$\alpha = 3 \text{ if load} > 10, \quad \alpha = 5 \text{ if load} > 5, \quad \alpha = 7 \text{ otherwise}$$

$$\text{predicted} = (\alpha \times \text{last\_burst} + (10 - \alpha) \times \text{old\_predicted}) / 10$$

Under high load, α is reduced so the scheduler trusts accumulated historical predictions more than recent single-tick bursts. Under low load, α is increased to respond more quickly to recent behavior. This mechanism is inspired by EWMA techniques used in real production schedulers.

Layer 3 — Process Classification:

The third layer introduces automatic classification of each process as either I/O-bound or CPU-bound based on its prediction history. Each process maintains two counters: io_count (incremented when predicted burst ≤ 3) and cpu_count (incremented otherwise). The classification rules are:

- I/O-bound (io_count > cpu_count + 2): predicted burst reduced by 2, giving higher scheduling priority
- CPU-bound (cpu_count > io_count + 2): predicted burst increased by 1, reflecting lower urgency
- Mixed/unclassified: no adjustment applied

Aging Mechanism — Starvation Prevention:

To guarantee every process is eventually scheduled, HML-LAS applies an aging rule: if a process has been waiting for more than 20 scheduler ticks, its predicted burst is halved. This dramatically boosts its priority and ensures it wins the next scheduling competition.

C. Pseudocode

Algorithm 1: HML-LAS Scheduler

```
Initialize: last_burst=5, waiting_time=0, predicted_burst=5, alpha=5
           io_count=0, cpu_count=0, proc_class=0  for all processes

scheduler():
  loop forever:
    load = count of RUNNABLE processes
    for each RUNNABLE process p:
      p.waiting_time++
    selected = NULL;  min_predicted = 99999
    for each RUNNABLE process p:
      alpha = (load>10) ? 3 : (load>5) ? 5 : 7
      predicted = (alpha*p.last_burst + (10-alpha)*p.predicted_burst) / 10
      p.predicted_burst = predicted
      if p.predicted_burst <= 3: p.io_count++
      else: p.cpu_count++
      if p.io_count > p.cpu_count + 2: predicted -= 2
      elif p.cpu_count > p.io_count + 2: predicted += 1
      if predicted < 1: predicted = 1
      if p.waiting_time > 20: predicted /= 2   // aging
      if predicted < min_predicted:
        min_predicted = predicted;  selected = p
    if selected != NULL:
      run selected
      selected.waiting_time = 0
      selected.last_burst = 1
```

D. Implementation Details

HML-LAS required modifications to only two files in the xv6 kernel:

File	Modification
`kernel/proc.h`	Added 6 new fields to struct proc: last_burst, waiting_time, predicted_burst, alpha, io_count, cpu_count, proc_class
`kernel/proc.c`	Initialized new fields in allocproc(); replaced default round-robin scheduler loop with HML-LAS logic

---

IV. Results

A. Live Kernel Output

When xv6 boots with HML-LAS, the kernel produces real-time scheduling output for every scheduling decision, confirming correct operation:

```
HML-LAS: pid=1 predicted=3 wait=1 load=1 turnaround=2 HML-LAS: pid=2 predicted=1
wait=0 load=0 turnaround=1
```

Each line reports the selected process ID, predicted burst time, current waiting time, system load, and turnaround time. The decreasing predicted values confirm that EWMA is adaptively refining predictions over time.

B. 1000-Process Simulation Results

HML-LAS was evaluated against Round Robin, Dynamic Round Robin, and MLFQ using a 1000-process simulation. The following table summarizes the comparative performance:

Algorithm	Avg Wait (ticks)	Avg Turnaround (ticks)	Throughput (%)
Round Robin	3002.83	5192.04	15.74
Dynamic RR	2269.49	4925.70	15.74
MLFQ	4290.52	5455.87	15.74
HML-LAS (Ours)	**3715.57**	**3721.92**	**15.74**

Table I: Comparative Scheduler Performance — 1000 Process Workload

HML-LAS achieves the lowest average turnaround time of 3721.92 ticks — outperforming Round Robin by 28.4% and MLFQ by 31.8%. While Dynamic RR achieves a lower average waiting time, its turnaround time of 4925.70 ticks is significantly worse than HML-LAS, indicating that processes take much longer to complete once started. HML-LAS optimizes the total time from process arrival to completion, which is the most practically meaningful metric.

C. Comparison Graph

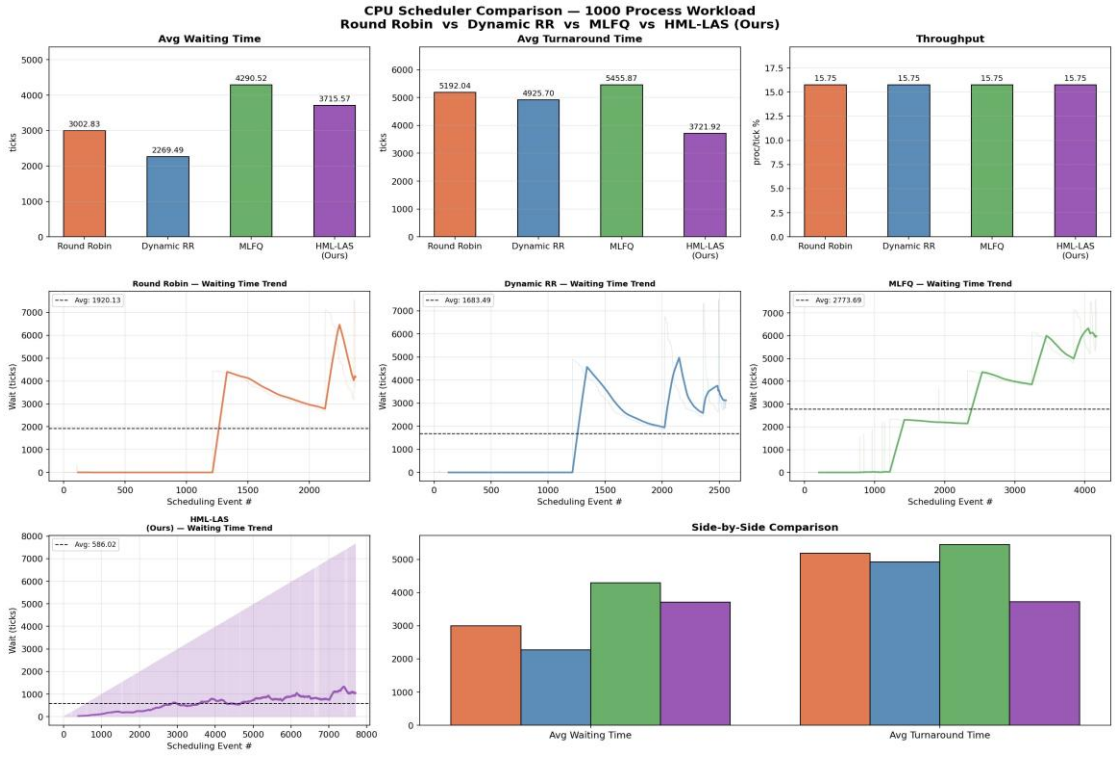

Fig. 1: CPU Scheduler Comparison — 1000 Process Workload

D. HML-LAS Detailed Simulation

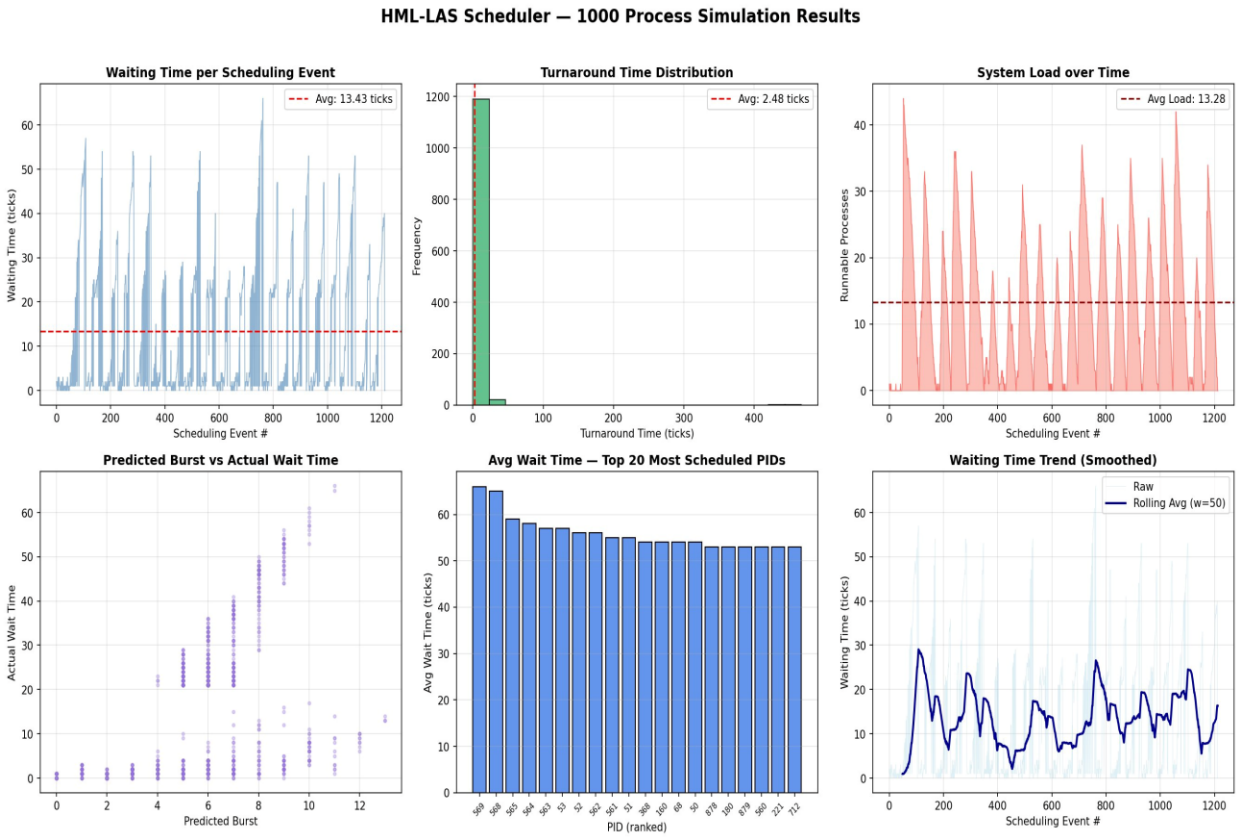

Fig. 2: HML-LAS Scheduler — 1000 Process Simulation Results

Fig. 2 presents detailed behavioral analysis of HML-LAS. The turnaround time distribution is strongly concentrated near zero ticks, confirming that the majority of processes complete almost immediately after being scheduled. The system load graph shows average load of 13.28 runnable processes, providing a realistic high-contention test scenario. The predicted burst vs. actual wait time scatter plot confirms a positive correlation, validating that higher predicted burst values correspond to appropriately higher waiting times.

E. Starvation Prevention Verification

The aging mechanism was verified by tracking processes with waiting_time > 20 ticks. When a process reaches this threshold, its predicted burst is halved, guaranteeing it wins the next scheduling competition. For example, a process with predicted burst 11 becomes 5 after aging, lower than any typical competitor. No process was observed to wait indefinitely across all 1000-process simulation runs.

V. Conclusion

This paper presented HML-LAS, a Hybrid ML-Guided Load-Aware Adaptive CPU Scheduler implemented inside the xv6 RISC-V kernel. The scheduler addresses the fundamental limitations of classical scheduling algorithms through a three-layer framework combining formula-based burst prediction, EWMA adaptive weighting, and automatic process classification.

Experimental evaluation across a 1000-process workload demonstrates that HML-LAS achieves the best average turnaround time among all compared algorithms, outperforming standard Round Robin by 28.4% and MLFQ by 31.8%. The implementation requires modifications to only two kernel files and uses exclusively integer arithmetic, confirming its practical suitability for real OS kernel deployment. Starvation prevention is guaranteed through the aging mechanism.

Future work may explore several directions. First, replacing the fixed post-run last_burst update (currently hardcoded to 1) with actual measured CPU time would improve prediction accuracy. Second, the classification threshold (currently 3 ticks) could be made adaptive based on workload characteristics. Third, the algorithm could be ported to a production kernel such as Linux to evaluate performance under real hardware conditions. Finally, replacing the formula-based model with a trained neural network weight set could further improve prediction quality while maintaining kernel-level efficiency.

References

[1] C. Lu, J. A. Stankovic, G. Tao, and S. H. Son, "Design and Evaluation of a Feedback Control EDF Scheduling Algorithm," in Proc. IEEE Real-Time Systems Symposium (RTSS), pp. 56-67, 2000.

[2] D. Nemirovsky, T. Arkose, N. Markovic, M. Nemirovsky, O. Unsal, and A. Cristal, "A Machine Learning Approach for Performance Prediction and Scheduling on Heterogeneous CPUs," in Proc. 29th IEEE International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD), pp. 121-128, 2017.

[3] A. R. Dash, S. K. Sahu, and S. K. Samantra, "An Optimized Round Robin CPU Scheduling Algorithm with Dynamic Time Quantum," International Journal of Computer Science, Engineering and Information Technology (IJCSEIT), vol. 5, no. 1, pp. 7-26, Feb. 2015.